TP 1 - Application Monolithique avec Spring Boot

Gestion de Produits Simple

Objectifs

- Créer une application Spring Boot monolithique simple
- Développer une API REST CRUD complète
- Comprendre l'architecture en couches (Entity, Repository, Service, Controller)
- Découvrir RestRepository pour simplifier le code
- Identifier les avantages et limites d'un monolithe

Outils Nécessaires

- JDK 17+
- Maven 3.8+
- IDE (IntelliJ, Eclipse ou VS Code)
- PostgreSQL ou MySQL
- Postman

Étapes du TP

Étape 1 : Initialisation du Projet

Sur https://start.spring.io, créez un projet avec :

- Spring Web, Spring Data JPA, PostgreSQL Driver, Lombok, Validation

Configuration ***application.properties*** :

```
spring.datasource.url=jdbc:postgresql://localhost:5432/ecommerce
spring.jpa.hibernate.ddl-auto=update spring.jpa.show-sql=true
```

Étape 2 : Créer l'Entité Product

```
package com.ecommerce.monolith.product.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;
import java.math.BigDecimal;

@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank(message = "Product name is required")
    @Column(nullable = false)
    private String name;
    @Column(length = 1000)
    private String description;
    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;
    @NotNull(message = "Stock is required")
    @PositiveOrZero(message = "Stock cannot be negative")
    @Column(nullable = false)
    private Integer stock;
}
```

Étape 3 : Créer le Repository

Version classique :

```
package com.ecommerce.monolith.product.repository;
```

```
import com.ecommerce.monolith.product.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    List<Product> findByNameContainingIgnoreCase(String name);

}
```

Version RestRepository (API automatique) :

```
@RepositoryRestResource(path = "products")
public interface ProductRepository extends JpaRepository<Product, Long> {
     List<Product> findByNameContaining(@Param("name") String name);
}
```

Avec @RepositoryRestResource, Spring génère automatiquement tous les endpoints REST !

Étape 4 : Créer le Service

```
package com.ecommerce.monolith.product.service;

import com.ecommerce.monolith.product.model.Product;
import com.ecommerce.monolith.product.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;

@Service
@RequiredArgsConstructor
@Transactional
public class ProductService {

    private final ProductRepository repository;

    @Transactional(readOnly = true)
    public List<Product> getAll() {
        return repository.findAll();
    }

    @Transactional(readOnly = true)
    public Product getById(Long id) {
        return repository.findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Product not found
with id: " + id));
    }

    public Product create(Product product) {
        return repository.save(product);
    }

    public Product update(Long id, Product details) {
        Product product = getById(id);

        product.setName(details.getName());
        product.setDescription(details.getDescription());
        product.setPrice(details.getPrice());
        product.setStock(details.getStock());

        return product; // save inutile si entité managed
    }

    public void delete(Long id) {
        Product product = getById(id);
        repository.delete(product);
    }
}
```

Étape 5 : Créer le Controller

```
package com.ecommerce.monolith.product.controller;

import com.ecommerce.monolith.product.model.Product;
import com.ecommerce.monolith.product.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;
import jakarta.validation.Valid;
import java.util.List;

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
@Validated
public class ProductController {

    private final ProductService service;

    @GetMapping
    public ResponseEntity<List<Product>> getAll() {
        List<Product> products = service.getAll();
        return ResponseEntity.ok(products);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getById(@PathVariable Long id) {
        Product product = service.getById(id);
        return ResponseEntity.ok(product);
    }

    @PostMapping
    public ResponseEntity<Product> create(@Valid @RequestBody Product product) {
        Product created = service.create(product);
        return ResponseEntity.status(HttpStatus.CREATED).body(created);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Product> update(@PathVariable Long id,
                                          @Valid @RequestBody Product product) {
        Product updated = service.update(id, product);
        return ResponseEntity.ok(updated);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

Étape 6 : Tester avec Postman

Méthode	URL	Description
GET	/api/products	Liste tous les produits
POST	/api/products	Créer un produit
PUT	/api/products/{id}	Modifier un produit
DELETE	/api/products/{id}	Supprimer un produit

Exercice

Ajoutez une entité Category et créez la relation ManyToOne avec Product. Testez avec RestRepository.